

Term Project — High-Doppler / High-CFO OFDM Receiver Design

Course: OFDM (Spring 2026) — Term Project

Weight: 60% of course grade (combined with the 40% Lec 10/11 presentations = 100%)

Format: Individual work — no groups

Submission deadline: June 11, 2026 at 23:59 (email to ecavus@aybu.edu.tr)

Project presentation: June 12, 2026 at 14:30 — in person or online, individual, ≈10 min talk + 5 min Q&A

Overview

You will design, implement and benchmark an OFDM receiver that recovers uncoded QPSK data from a **single OFDM frame (10 symbols)** of a realistic drone-radio waveform, in the presence of:

- a frequency-selective multipath channel (FPV_C drone profile, 4 taps),
- a Doppler spread (Jakes' model) that is **swept across** {0, 1000, 2000, 3000} Hz — 3000 Hz is the project operating point (FPV_C high mobility); the lower values are diagnostic so you can see how much of your residual BER is caused by mobility versus the other impairments,
- a residual **symbol timing offset (STO)** of up to ± 32 samples ($\sim 1 \mu\text{s}$),
- a residual **carrier frequency offset (CFO)** of up to ± 12 kHz, and
- AWGN at a sweep of E_b/N_0 values.

This term project builds directly on the **Lec 10/11 presentations**, where you learned (i) how STO, CFO and Doppler break OFDM orthogonality and (ii) the canonical correction methods. You are **completely free** in your choice of approach — pick any method from those presentations (M1–M4), combine several of them, adopt something from the literature, or invent your own. The only thing that matters is BER and complexity.

The student whose receiver achieves the lowest BER at the lowest operation count wins the highest grade.

Scope (what is in and what is out)

In scope	Out of scope
One OFDM frame (10 OFDM symbols, see frame table below)	Anything outside this frame
QPSK only , uncoded	Higher-order modulations, FEC / channel coding
Data subcarriers + pilot subcarriers	Preamble, synchronization training
FPV_C drone multipath channel	Other channel profiles
Custom STO + CFO + Doppler correction in the receiver	Modifying the channel
Symbol-level or slot-level interleaving (allowed — you may modify the TX for this)	Channel coding (FEC) of any kind
Pilot pattern rearrangement (allowed — TX modification permitted, max 64 pilot SCs per OFDM symbol)	Increasing the pilot count above 64 per symbol

You will receive the entire transmitter and channel as a single black-box MATLAB function. Your job is the receiver.

Frame parameters

The received signal is one OFDM frame with the following parameters:

Symbol	Description	Value
Fs	Sampling frequency	30.72 MHz
SCS	Sub-carrier spacing	30 kHz
N_FFT	FFT size	1024 samples
N_CP	CP length	288 samples
N_sym	OFDM symbols per frame	10
BW	Occupied bandwidth	10 MHz
Used SCs	Active subcarriers per OFDM symbol	312 (26 resource blocks × 12 SC)
DC SC	Not used	(subcarrier index 0 unused; ±1 ... ±156 used)
Pilots	Reserved as pilots per data symbol	64 sub-carriers (scattered, dense_4sym pattern)

All sizes you need to write code with are in **samples**. If you ever need a duration, divide by Fs (e.g., one OFDM symbol = N_FFT / F_s).

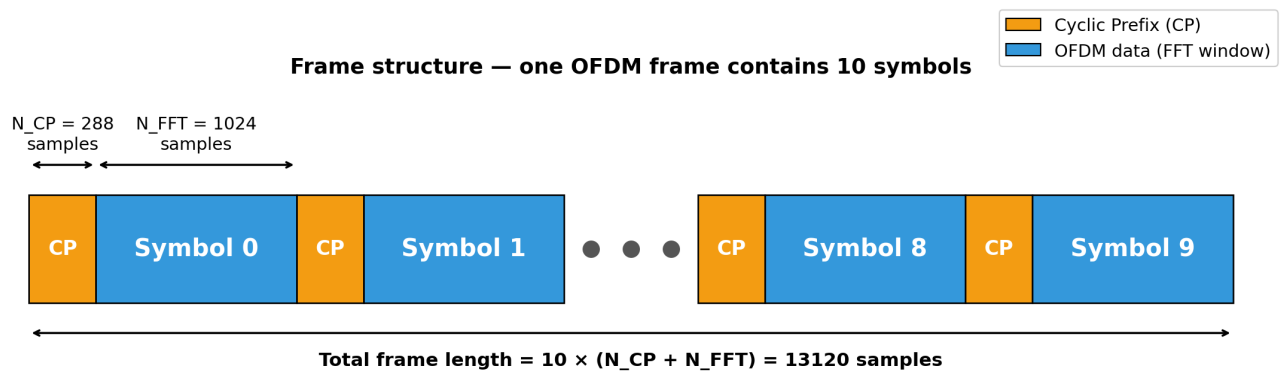


Figure 1: Frame structure — 10 OFDM symbols, each with a CP

Pilot pattern: **dense_4sym** — pilots at even SCs {0, 2, 4, 6, 8, 10} of each resource block at symbol indices {0, 3, 6, 9}.

The same dense_4sym tile **repeats identically across every one of the 26 resource blocks**, so the receiver's frequency-domain pilot grid is dense (156 pilot SCs on every pilot-bearing symbol) but its time-domain pilot grid is sparse (only 4 of the 10 OFDM symbols carry pilots, namely {0, 3, 6, 9}). The figure below shows that whole-frame view; the vertical ellipsis stands in for the 23 RBs (RB 3 ... RB 25) that we don't draw — they are bit-for-bit copies of the three RBs that are drawn.

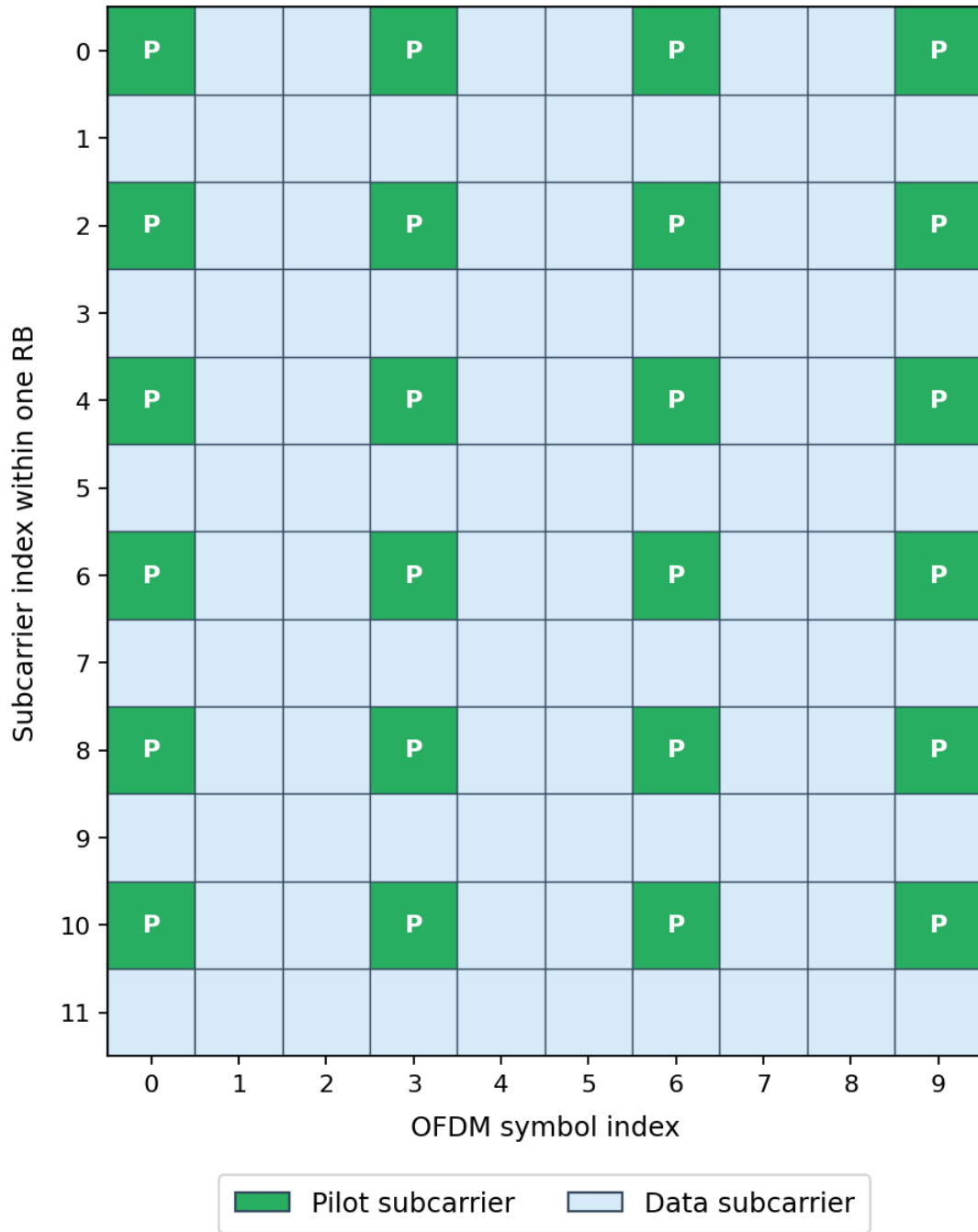
That frequency-dense / time-sparse asymmetry is what shapes the project: you have a sharp picture of $H[k]$ *only on 4 snapshots in time*, and the channel changes between those snapshots at $f_d = 3000$ Hz. Every Wiener / Kalman / decision-directed estimator you might add to `doppler_track.m` is fundamentally a strategy for filling the gap between those 4 snapshots — that gap is the irreducible Doppler floor you see in the baseline BER curve below.

Channel and impairments

Multipath: FPV_C urban/suburban drone profile

- 4 taps, ~12-sample maximum excess delay
- Modelled with `comm.RayleighChannel`, Jakes spectrum, swept over $f_d \in \{0, 1000, 2000, 3000\}$ Hz
- **$f_d = 3000$ Hz is the official operating point** (FPV_C drone, high mobility) and is the value the grading harness uses for the leaderboard score
- The lower Doppler values (0, 1000, 2000 Hz) are run as **diagnostic curves on the same axes** — they let you (and the grader) see how much of your residual BER comes from mobility versus the other impairments. $f_d = 0$ collapses the

Pilot pattern (dense_4sym) — one resource block of 12 subcarriers



Each of the 26 resource blocks (312 active subcarriers total) repeats this pattern.
 Pilots: 6 SCs $\times$ 4 symbols = 24 per RB $\rightarrow$ 624 pilots per frame.

Figure 2: Pilot pattern in one resource block (dense_4sym)

Pilot pattern across the full frame — 10 OFDM symbols × 26 resource blocks (dense_4sym)

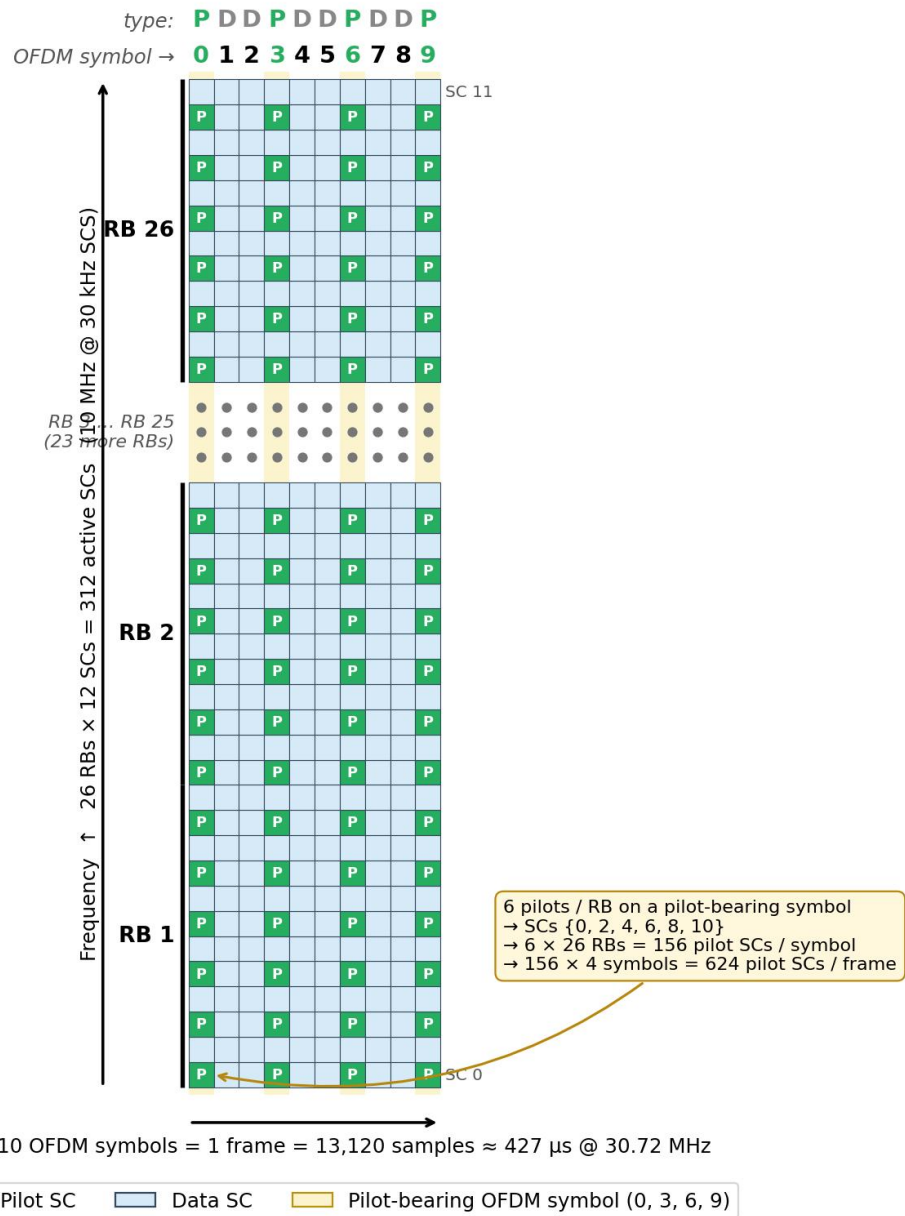


Figure 3: Pilot pattern across the full frame — 10 OFDM symbols × 26 resource blocks

Rayleigh process to a single static draw of the four taps and is the cleanest sanity check that the receiver chain itself is sound

- The TX exposes `fd_doppler` as an optional override: `ofdm_tx_and_channel(SNR_dB, seed, struct('fd_doppler', fd))`

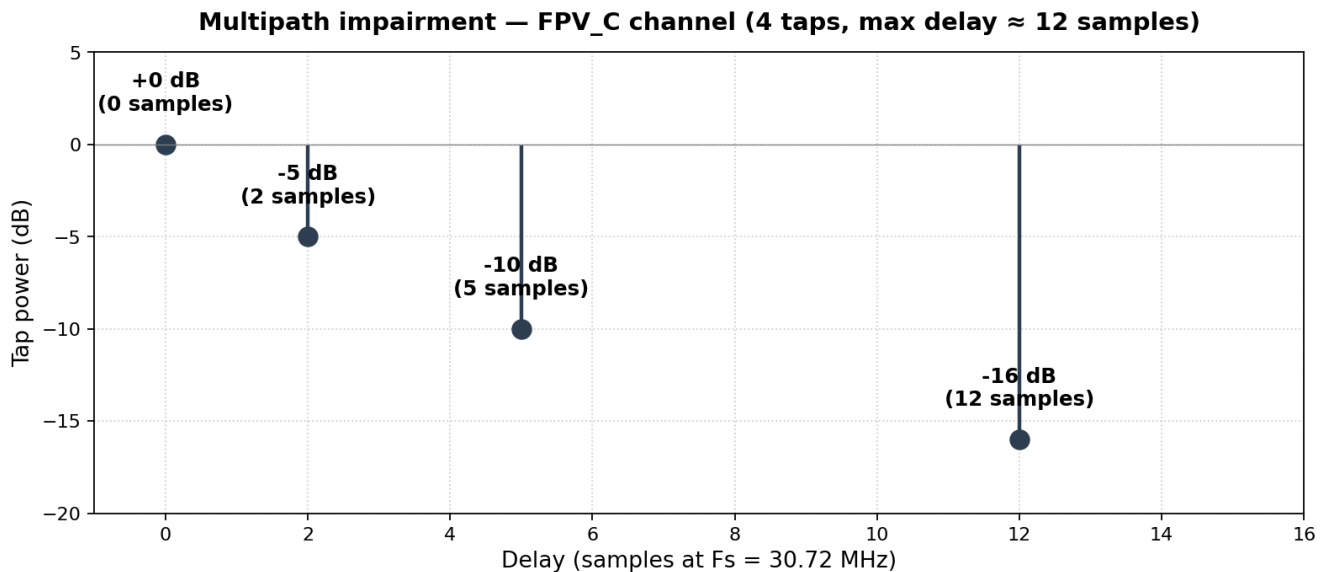


Figure 4: Multipath impairment — FPV_C channel impulse response

STO (symbol timing offset): up to ± 32 samples residual

- The instructor sets the actual value inside `tx_and_channel.m`. You do not pick it, you do not see it — your receiver must estimate it from the received signal.
- The value is always within the CP, so there is no ISI from the previous symbol. The visible effect is a **linear phase ramp across subcarriers** (the “ladder” signature from Lec 9).
- **Assumption:** “The pre-correction stage upstream of the data frame is assumed to reduce the timing uncertainty to within ± 32 samples. The residual STO must be estimated and removed by the receiver — either explicitly (e.g., the Lab 9 phase-slope estimator) or implicitly (absorbed into the channel estimate).”

STO impairment — receiver's FFT window starts δ samples earlier, inside the CP

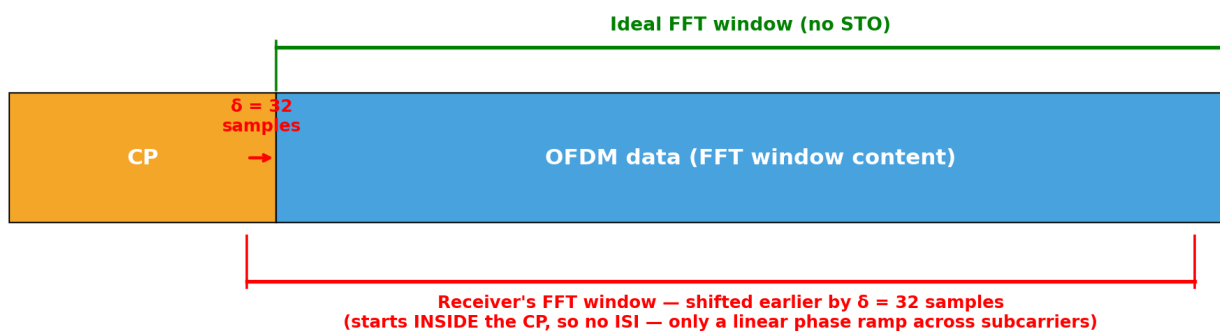


Figure 5: STO impairment — receiver's FFT window starts δ samples off

CFO (carrier frequency offset): up to ± 12 kHz residual

- The instructor sets the actual value inside `tx_and_channel.m`. You do not pick it, you do not see it — your receiver must estimate it from the received signal.

- The value is constant throughout the frame.
- **Assumption:** “In a full system the control-slot preamble would reduce the raw ± 15 kHz HW oscillator offset down to $\sim \pm 1.5$ kHz before the data frame begins. In this project we skip the preamble and operate on the data frame alone — the receiver therefore faces the full ± 12 kHz HW CFO and must estimate and correct it from the data frame itself.”

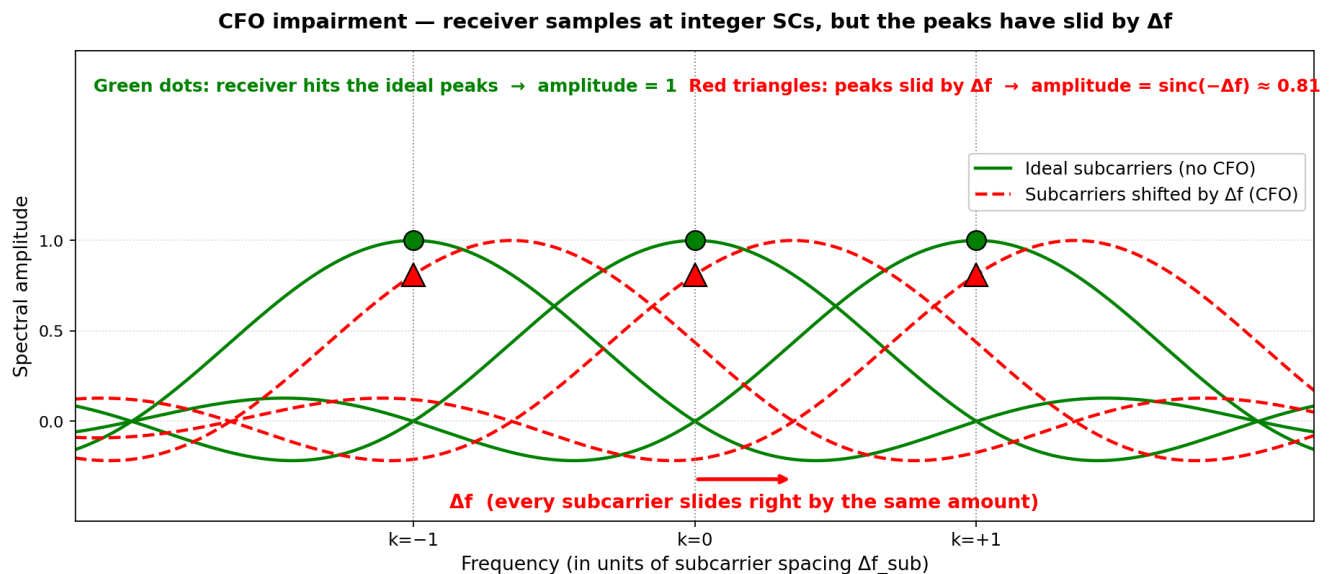


Figure 6: CFO impairment — every subcarrier shifts by Δf

AWGN: added at a configurable E_b/N_0 ; evaluation sweeps E_b/N_0 from 0 to 30 dB.

How the BER sweep works (Monte Carlo)

You **do not** write the SNR-sweep loop yourself. Our grading harness (`evaluate.m`) does it: at each E_b/N_0 point it calls `tx_and_channel(SNR_dB, seed)` to generate one received frame, calls your `student_rx(rx_samples, params)` to get the recovered bits, compares against the ground truth, and accumulates errors.

At each E_b/N_0 point the harness runs adaptively — it keeps generating new frames (each with a different seed) until either of two stopping conditions is hit:

- ≥ 200 bit errors have been collected (the usual Monte Carlo target for $\pm 10\%$ BER confidence), or
- 5 000 frames have been simulated (a hard upper bound to keep the sweep finite).

Whichever happens first ends that E_b/N_0 point. This wastes no compute at low SNR (where errors pile up fast) and gives enough statistical resolution at high SNR (where a strong receiver can drive BER below 10^{-4}).

For **your own** sweeps, you don't have to match the grading harness exactly. Two regimes to keep in mind:

- **While developing / iterating:** if simulations are taking too long, feel free to lower the frame count (e.g., `N_trials = 20–50` per point). The curves will be noisy but good enough to see whether a change in your algorithm helped or hurt. Speed wins over precision during the iteration loop.
- **For the final BER plot you submit:** the curve must be **smooth**. Use enough frames per E_b/N_0 point to collect at least ~ 100 bit errors at every point on the sweep — the rule of thumb is the same ~ 200 errors / 5 000 frames cap the grader uses. A noisy or jumpy curve at high SNR will be obvious in the PNG and will hurt your “BER plot quality” impression.

The starter pack will include a `run_ber_sweep.m` example that defaults to the development setting; bump up `N_trials` (or the per-point error target) before generating your final plot.

Doppler grid. `run_ber_sweep.m` ships with an outer loop over `DOPPLER_GRID = [0, 1000, 2000, 3000]` Hz and produces one BER curve per value on the same axes. The final plot you submit must contain **all four curves**. The leaderboard

ranking uses the $f_d = 3000$ Hz curve (the project operating point); the other three are there so you can argue *why* your design behaves the way it does — a receiver that hugs the AWGN floor at $f_d = 0$ but flares out at $f_d = 3000$ Hz has a Doppler-tracking problem, whereas one that is flat-but-high across all four Doppler values has an STO/CFO problem.

Tier-1 baseline reference curves. Running the starter `run_ber_sweep.m` as shipped (with the minimum CP-self-correlation STO + CFO baselines that we provide in the stubs, and the LS + nearest-neighbour channel estimator + ZF equalizer for everything else) produces the figure below. *This is the curve your receiver has to beat at $f_d = 3000$ Hz* — every point on your $f_d = 3000$ Hz curve that lies below the purple baseline is a point you score on.

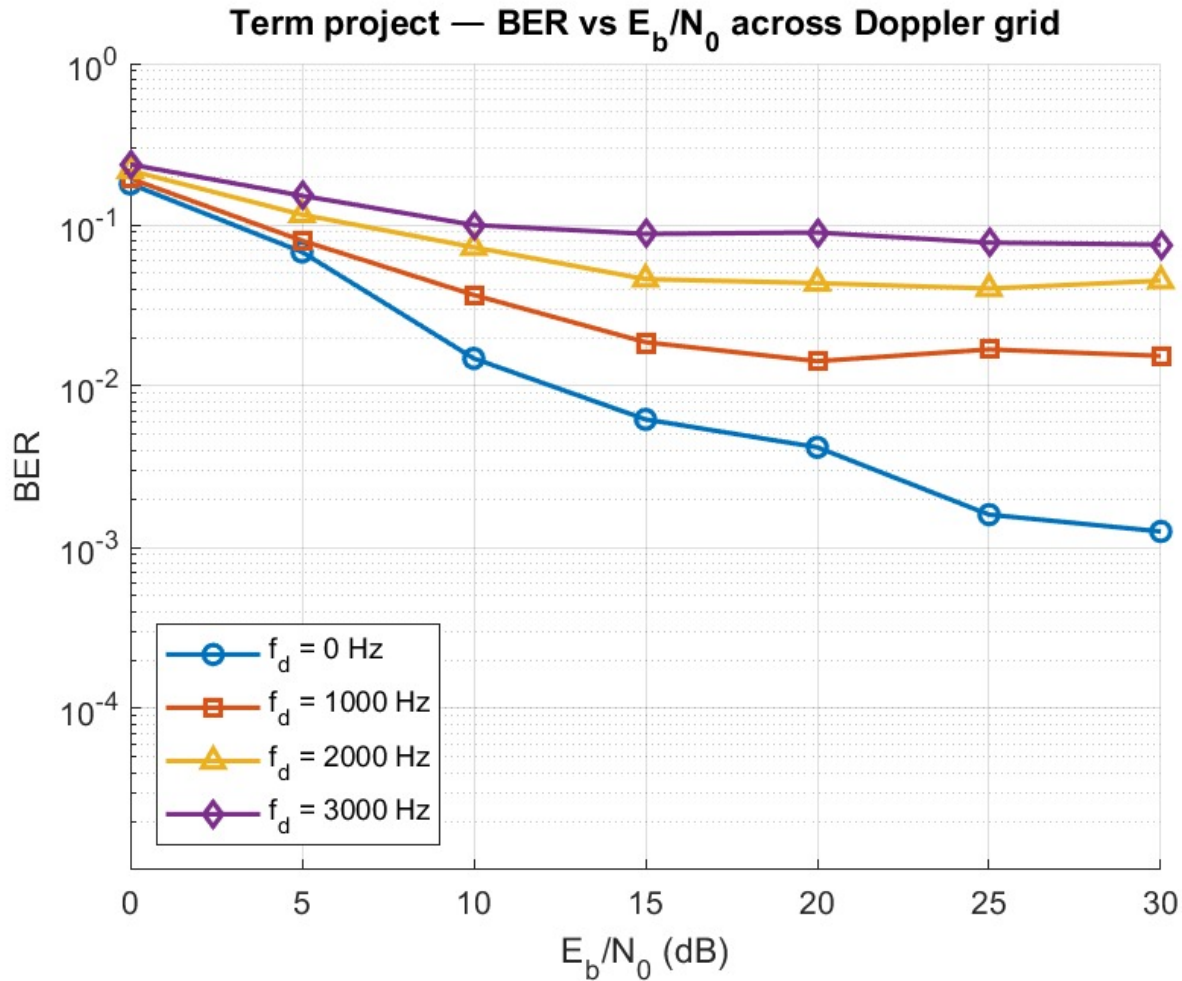


Figure 7: Tier-1 baseline BER vs E_b/N_0 for $f_d \in \{0, 1000, 2000, 3000\}$ Hz

How to read it:

- $f_d = 0$ (blue): the static frequency-selective channel. The minimum STO+CFO baseline + LS channel estimate is essentially complete here, so the curve keeps falling to $\sim 10^{-3}$ at 30 dB. There is little room left to improve at $f_d = 0$.
- $f_d = 1000$ / 2000 / 3000 (orange / yellow / purple): each settles into its own irreducible **Doppler error floor** at high SNR ($\sim 1.5\%$, $\sim 4.5\%$, $\sim 7.5\%$). The floor grows with f_d because the LS-at-pilots + nearest-neighbour-in-time channel estimate goes stale faster as the channel changes within the frame. **The entire residual at high SNR is the channel-tracker — that is where the biggest BER gain in this project lives.**

What we give you (starter/ folder)

Four MATLAB files (to be released alongside this README — separate task):

File	Purpose
<code>tx_and_channel.m</code>	Generates one received OFDM frame. Signature: <code>[rx_samples, params, ref_bits] = tx_and_channel(SNR_dB, seed)</code> . Output: complex baseband, frame params, ground-truth bits. The internal chain is shown below.
<code>ofdm_rx.m</code>	Tier-1 baseline orchestrator. Calls every sub-function below in order. You don't have to modify this file directly — instead replace the relevant sub-function.
<code>cp_remove.m</code> , <code>ofdm_demod.m</code> , <code>channel_estimate.m</code> , <code>equalize.m</code> , <code>qpsk_demap.m</code> <code>sto_estimate_correct.m</code> , <code>cfo_estimate_correct.m</code> , <code>doppler_track.m</code>	Given (replaceable). Working implementations of the standard blocks: CP removal, FFT, LS-at-pilots + nearest-neighbour-in-time channel estimate, zero-forcing one-tap equalizer, hard-decision QPSK demap. Stubs (you implement). Each file currently passes its input through unchanged. With these stubs in place the chain runs end-to-end but the BER is essentially random (~40 % at $E_b/N_0 = 28$ dB) because STO and CFO are uncorrected. This is the <i>starting</i> point you have to improve.
<code>params_data.m</code>	Frame parameter struct. Used by both the TX and your RX.
<code>run_ber_sweep.m</code>	Example BER vs E_b/N_0 sweep harness — drop-in script you can adapt.

No external dependencies beyond MATLAB R2023b+ and the Communications Toolbox (`comm.RayleighChannel`).

Reproducibility: every `(SNR_dB, seed)` pair produces the exact same received signal across all students.

The TX chain inside `tx_and_channel.m`:

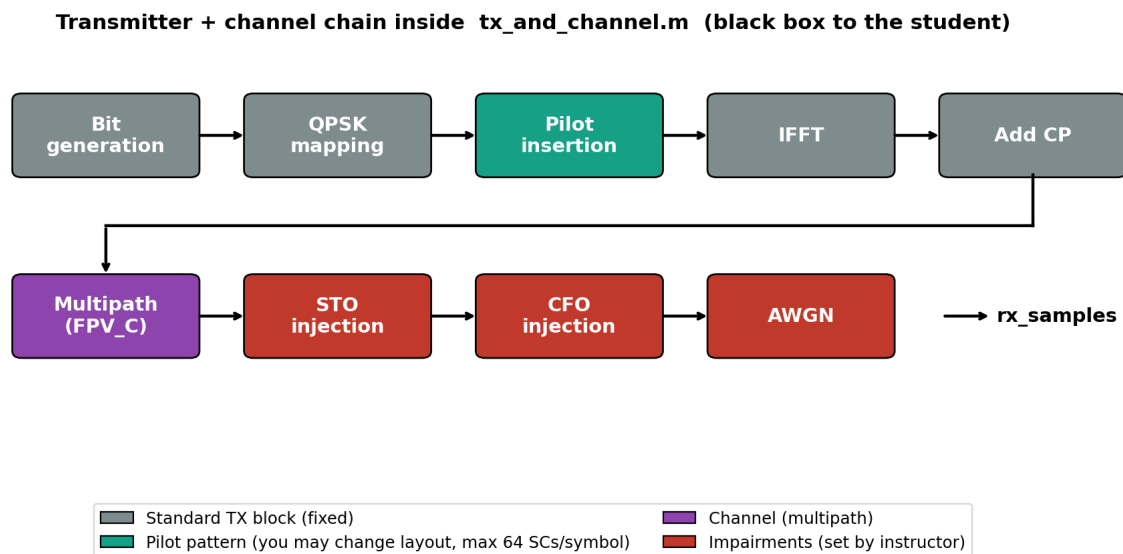


Figure 8: Transmitter and channel chain — what `tx_and_channel.m` does internally

What you build

Your `student_rx(rx_samples, params)` function must beat the Tier-1 baseline. The receiver pipeline:

The interesting parts — where the project is actually won — are:

1. **Residual STO estimation and correction.** The ± 32 -sample timing offset produces a linear phase ramp across subcarriers (the “ladder” signature from Lec 9). Two ways to address it: estimate the slope explicitly with the Lab 9 `unwrap` + `polyfit` method and remove it, *or* let a robust channel estimate absorb it. Either path is valid — pick what’s cheapest.

Receiver chain — `student_rx(rx_samples, params) → bits`

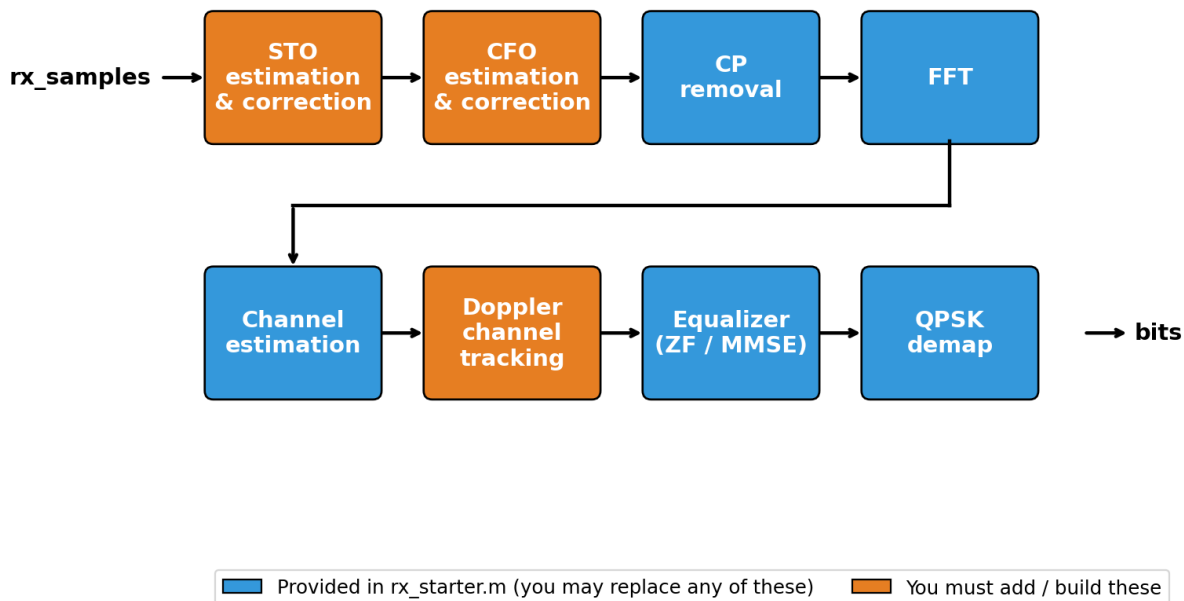


Figure 9: Receiver chain — `student_rx(rx_samples, params) → bits`

2. **Residual CFO estimation and correction.** The 12 kHz HW CFO rotates every sample. Without correction, every OFDM symbol picks up a different phase, and the LS channel estimate goes stale almost immediately. *Lec 10/11 methods M1–M3 all give you tools to attack this.*
3. **Doppler-aware channel tracking.** At $f_d = 3000$ Hz the channel evolves *within* the frame (~30% of coherence time). A one-shot LS estimate at the first pilot symbol is wrong by the last data symbol. You need to track $H[k]$ across the 10 OFDM symbols. The starter code provides a **standard LS channel estimator with nearest-neighbour time interpolation** — feel free to replace it with a smarter estimator (Wiener filtering, decision-directed update, Kalman, polynomial fit across symbols, etc.).
4. **(Optional, for top grades) ICI-aware equalization.** When $T_{\text{sym}} \cdot f_d$ is non-negligible, neighbouring subcarriers leak into each other. A simple banded (3-diagonal) equalizer can recover some of that — method M4 from Week 2.

You may also modify **any of the starter blocks** — CP removal, FFT, channel estimation, equalizer, QPSK demap — if you want. The starter code only gives one *standard* implementation of each block; you are not obligated to keep it.

Interleaving is allowed. Channel coding (FEC) is *not* allowed, but you may add **symbol-level or slot-level interleaving** to spread the impact of in-frame fades. If you choose to use interleaving, you may modify the transmitter side of `tx_and_channel.m` to insert the matching interleaver — this is the only TX modification permitted. Document any TX change clearly in your 1-page report.

Method choice is open. Use any method from the Lec 10/11 deck (M1–M4), any combination of them, anything you find in the literature, or your own design. We will not judge your choice — only the resulting BER and the operation count.

Pilot pattern is open too. The starter ships with the `dense_4sym` pattern (64 pilot SCs at symbols {0,3,6,9}, even SCs of each RB). If you believe a different pattern (e.g., comb pilots on every symbol, scattered/diagonal pattern, etc.) helps your channel-tracking algorithm, you may rearrange it — **as long as the total stays at 64 pilot SCs per OFDM symbol**. This is the only other TX modification permitted; document any change in your 1-page report.

You are **not allowed** to share code with other students or use another student's submission as a starting point.

Deliverables

Submit **three files** by email to ecavus@aybu.edu.tr, deadline **June 11, 2026 at 23:59**:

1. `<lastname>_starter.zip` — a zip of your *entire* modified `starter/` folder. The grader will unzip it, put it on the MATLAB path, and call `ofdm_rx(rx_samples, params)`. Your modifications can live in any of the `.m` files; everything we shipped is yours to edit.
2. `<lastname>_report.pdf` — 1-page PDF. Must include:
 - which methods (M1–M4 or your own variant) you implemented and *why*;
 - one paragraph on what worked and what didn't;
 - your operation-count estimate from MATLAB Profiler.
3. `<lastname>_ber.png` — your BER vs E_b/N_0 plot. Must contain one curve per Doppler value in the $\{0, 1000, 2000, 3000\}$ Hz grid (your receiver), plus the Tier-1 baseline at $f_d = 3000$ Hz on the same axes for reference.

File-naming is strict — submissions that don't follow it lose 5 points.

Project presentation — June 12, 2026 at 14:30 (in person or online)

On the day after the submission deadline (**Friday, June 12, 2026 at 14:30**) every student presents their work. You may attend in person or join online — both modes count the same. The presentation is short (≈ 10 min talk + 5 min Q&A) and is graded as part of the term project.

What you will be asked. Not “what did you do” — *why* did you do it. You should be able to answer:

- *Why* did the Tier-1 baseline fail on this channel? Which impairment dominated *your* receiver's residual errors?
- *Why* did you pick the specific STO / CFO / Doppler-tracking method you implemented? What were the alternatives and what made yours better?
- If we changed Δf to 20 kHz instead of 12 kHz, *which* of your blocks would break first, and how would you fix it?
- Walk me through one line of your code that you wrote and explain what it computes — physically, not syntactically.

Internalise, don't memorise. A rehearsed script that you can't deviate from will be obvious in 30 seconds. The point is to convince us that *you* designed and understood the receiver — not that you can recite a polished narrative. Be ready to draw on a whiteboard, modify a line of code on the spot, or sketch what would happen if a parameter changed.

If the algorithm came from a paper or a Lec 10/11 method, that's fine — *explain it in your own words and own it*.

Grading

Component	Weight	How it is measured
BER performance	50%	Your receiver is run by an evaluation harness over the official E_b/N_0 sweep (0 to 30 dB) at $f_d = 3000$ Hz with a held-out random seed (adaptive: ≥ 200 errors OR 5000 frames per point). The leaderboard score = how much you beat the Tier-1 baseline at the operating point. The other Doppler values ($\{0, 1000, 2000\}$ Hz) are required on the submitted plot but do not affect the score; they are used in the presentation Q&A to probe whether you understand which impairment is dominating your residual BER. Lower BER $\rightarrow$ higher score.
Complexity	20%	MATLAB Profiler operation count (calls to <code>+</code> , <code>*</code> , <code>fft</code> , <code>\</code> , etc.) on one frame. Lower op count $\rightarrow$ higher score. Prevents brute-force “throw compute at the problem” wins.
Project presentation (June 12, 14:30, in person or online)	30%	Depth of understanding, not rehearsed delivery. Graded on whether you can explain <i>why</i> your design choices work, defend them under Q&A, and adapt on the spot to “what if we changed X?” questions.

Term project total = 60% of course grade. Together with the 40% from Lec 10/11 presentations, this is the entire course grade — there is no separate exam.

Grades are assigned based on the combined score — higher combined scores receive higher grades. Absolute rubric, no quota.

Rules

1. **Individual work only.** No group submissions. Discussing high-level approaches with a classmate is fine; sharing code is not.
 2. **Cite everything you borrow.** Textbook formulas, papers, prior Week 2 method talks, online references — all must be cited. Failing to cite = academic dishonesty.
 3. **Reproducibility.** Your code must produce the same output for the same (SNR_dB, seed). Don't seed against the wall clock.
 4. **No toolbox dependencies beyond Communications Toolbox.** No 5G Toolbox, no LTE Toolbox.
 5. **No internet calls, no shell-outs.** Pure MATLAB.
-

Suggested workflow

Phase	What to do
1 — Reproduce the baseline	Run <code>run_ber_sweep.m</code> as shipped. With the pass-through stubs you should see $\text{BER} \approx 4 \times 10^{-1}$ (~40 %) at $E_b/N_0 = 28$ dB — random output, because STO and CFO are uncorrected. Confirm you can run the chain end-to-end before changing anything.
2 — Address residual STO	Look at the LS channel-estimate phase across subcarriers — you'll see a linear ramp. Either estimate the slope (Lab 9 method) and remove it, or trust your channel estimate to absorb it.
3 — Add residual CFO correction	Pick <i>one</i> M1–M3 method from your Week 2 work. Estimate the 12 kHz CFO from pilots (or CP). Apply correction. Re-measure BER.
4 — Add Doppler-aware channel tracking	Interpolate the LS pilot estimates across symbols (linear is enough; spline/Wiener is better). Re-measure BER.
5 — Replace the channel estimator if it pays off	The starter's LS + nearest-neighbour estimator is a baseline, not a ceiling. If your BER is stuck above the AWGN floor, swap in a smarter channel estimator (Wiener filtering, decision-directed update, polynomial fit across symbols, Kalman tracker, MMSE with known channel statistics, ...). This is often where the biggest BER gain comes from.
6 — (Optional) ICI-aware equalizer	If you have time, replace the ZF one-tap with a 3-diagonal MMSE. Measure both BER and op-count cost. Keep the version with the better combined score.
7 — Write the 1-page report	Document what you did, what worked, what didn't. Run the MATLAB Profiler. Save your final BER plot.
8 — Sanity-check the submission	Open a fresh MATLAB session, copy only <code>starter/</code> and your <code>student_rx_<lastname>.m</code> to a clean folder, and run end-to-end. If it doesn't work in 5 minutes, the grader's harness won't either.

Folder layout

Term_Project/	
Term_Project_Description.md	← this file
starter/	← released alongside this document
params_data.m	← parameters (don't change)

<code>ofdm_tx_and_channel.m</code>	← black-box TX + channel + impairments
<code>ofdm_rx.m</code>	← receiver orchestrator (calls everything below)
<code>run_ber_sweep.m</code>	← example BER vs Eb/N0 harness
<code>cp_remove.m</code>	← given (replaceable)
<code>ofdm_demod.m</code>	← given (replaceable)
<code>channel_estimate.m</code>	← given (replaceable)
<code>equalize.m</code>	← given (replaceable)
<code>qpsk_demap.m</code>	← given (replaceable)
<code>sto_estimate_correct.m</code>	← STUB - you implement
<code>cfo_estimate_correct.m</code>	← STUB - you implement
<code>doppler_track.m</code>	← STUB - you implement

Questions

For *clarification* questions (about the starter code or the rules): email ecavus@aybu.edu.tr.

For *implementation* questions (which method to pick, why is my BER stuck): you are on your own — that's the project.